Introduction to Python Programming

What is Python ?

Python is a high-level, interpreted, general-purpose programming language. It is designed to be simple, readable, and easy to learn, which makes it popular among beginners as well as professional developers.

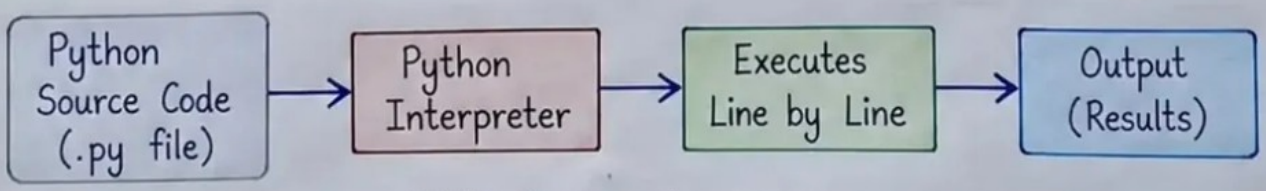
History of Python

- Python was created by Guido van Rossum in 1991.
- The first version of Python was released in 1991.
- It was developed at the National Research Institute for Mathematics and Computer Science (CWI) in the Netherlands.
- The latest stable version is Python 3.x.

Key Features of Python

- Simple and easy to learn
- Readable syntax
- High-level language
- Interpreted language
- Dynamically typed
- Object-oriented
- Portable (cross-platform)
- Large standard library
- Extensive community support

Python Execution Flow



@Mastercode.sagar

Introduction to Python Programming

History of Python

- Python was created by Guido van Rossum in 1989, and the first official release was in 1991.
- It was developed at the National Research Institute for Mathematics and Computer Science (CWI) in the Netherlands.
- Python was designed as an easy to read and maintain language.
- The name "Python" was inspired by the British comedy show Monty Python's Flying Circus.
- Over time, Python has evolved through many versions:

Version	Release Year	Key Points
Python 1.0	1991	First official release.
Python 2.0	2000	Major new features, list comprehensions.
Python 2.7	2010	Last release in 2.x series.
Python 3.0	2008	Introduced major changes, not fully backward compatible.
Python 3.x	2010 - Present	Regular updates with new features and improvements.

Why Python is Popular?

- Simple and clean syntax.
- Easy to learn for beginners.
- Powerful standard library.
- Used in multiple fields like web development, data science, machine learning, automation, etc.
- Large community support.

@Mastercode.sagar

Introduction to Python Programming

Applications of Python

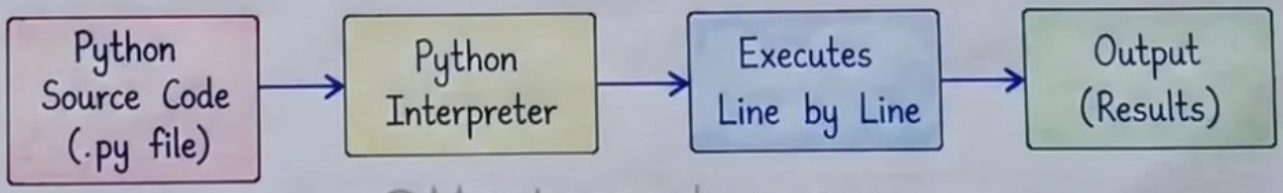
Python is a versatile language and is used in many fields. Some common applications are:

1. Web Development – Using frameworks like Django, Flask.
2. Data Science – Data analysis, visualization and machine learning.
3. Artificial Intelligence – Developing AI and deep learning models.
4. Automation – Automating repetitive tasks and scripting.
5. Web Scraping – Extracting data from websites.
6. Game Development – Using libraries like Pygame.
7. Desktop Applications – Building GUI applications.
8. Network Programming – Developing network tools and servers.
9. Cyber Security – Scripting and security tools.
10. Scientific Computing – Used in research and simulations.

Python vs Other Programming Languages

Feature	Python	C	Java
Syntax	Simple and easy	Complex	Moderate
Type	Interpreted	Compiled	Compiled
Speed	Slower than C/Java	Fast	Fast
Readability	Very high	Difficult	Moderate
Use Case	General-purpose	System programming	Enterprise applications
Learning Curve	Easy	Difficult	Moderate

Python Program Execution



@Mastercode.sagar

Introduction to Python Programming

Python Installation and Setup

- Python can be easily installed on Windows, macOS and Linux.
- The official website to download Python is www.python.org.
- Python comes with an interactive shell (IDLE) and can also be used with code editors like VS Code, PyCharm, etc.
- After installation, we can write and run Python programs in two ways:
  1. Interactive Mode (Python Shell)
  2. Script Mode (Using .py file)

Steps to Install Python (Windows)

1. Go to www.python.org
2. Download the latest version.
3. Run the installer.
4. Check the box "Add Python to PATH".
5. Click on "Install Now".
6. After installation, open Command Prompt and type:
   `python --version`
   to check the installed version.

Python Environments

Environment	Description	Usage
IDLE	Built-in Python IDE	Simple programs, good for beginners
VS Code	Popular code editor	Used with Python extension
PyCharm	Advanced IDE by JetBrains	Used for large projects
Jupyter Notebook	Web-based environment	Used in data science and ML

Checking Installation

```
C:\> python --version
Python 3.12.0
```

@Mastercode.sagar

Introduction to Python Programming

First Python Program

A simple Python program is used to print a message on the screen.

Program :

```
print("Hello, World!")
```

Output :

```
Hello, World!
```

Explanation :

- print() is a built-in function in Python.
- It displays the given message on the screen.
- Text inside double quotes (" ") is called a string.
- Each statement in Python is written in a new line (no semicolon is required).

Rules for Writing Python Program

1. Python is case-sensitive (print and Print are different).
2. Use proper indentation.
3. Use meaningful variable and function names.
4. String values are written inside single (' ') or double (" ") quotes.
5. No semicolon (;) is required at the end of the line.
6. Each statement is written in a new line.
7. Comments are written using #.

Multiple Statements

We can write multiple statements one after another.

```
print("Hello, World!")
print("Welcome to Python")
```

Output :

```
Hello, World!
Welcome to Python
```

@Mastercode.sagar

Introduction to Python Programming

Interactive Mode (Python Shell)

- After installing Python, we can use the interactive shell (IDLE) to write and execute Python code line by line.
- It is useful for testing small pieces of code.
- To open Python shell, search for "IDLE" in start menu and run it.
- The prompt >>> indicates that Python is ready to execute your code.

Example (Interactive Mode)

```
>>> print("Hello, World!")
Hello, World!
>>> 5 + 3
8
>>> 10 * 2
20
```

Script Mode (Using .py file)

- In script mode, we write the Python code in a file with .py extension and then run it.
- This is used for large programs.
- Steps:
  1. Open a text editor (Notepad, VS Code, PyCharm, etc.).
  2. Write the Python code.
  3. Save the file with .py extension (e.g. hello.py).
  4. Run the file using Python interpreter.

Example (Script Mode)

```
# File: hello.py
print("Hello, World!")
```

Run in terminal:

```
C:\> python hello.py
Hello, World!
```

@Mastercode.sagar

Introduction to Python Programming

Python Comments

- Comments are used to explain code and make it more readable.
- Comments are ignored by the Python interpreter.
- Single line comment is written using #.
- Python also supports multi-line comments using triple quotes (''' ''' or """ """).

Example (Single Line Comment)

```
# This is a comment
print("Hello, World!")
```

Example (Multi-line Comment)

```
'''
This is a multi-line comment.
It can span multiple lines.
'''
print("Python")
```

Python Indentation

- Indentation is used to define a block of code in Python.
- Python uses indentation instead of curly braces { }.
- Usually 4 spaces are used for indentation.
- All statements in the same block must have the same indentation.

Example (Indentation)

```
if 5 > 3:
    print("5 is greater than 3")
```

Python Statements and Line Structure

- Each statement in Python is written in a new line.
- Multiple statements can be written in a single line using a semicolon (;).
- But it is recommended to write each statement in a separate line for better readability.

Example (Single Line Multiple Statements)

```
a = 5 ; b = 10 ; print(a + b)
```

Output :

```
15
```

@Mastercode.sagar

Introduction to Python Programming

Variables in Python

- A variable is used to store data in memory.
- In Python, we do not need to declare the data type of a variable.
- The data type is automatically determined by Python.
- A variable can store numbers, text, or any other data.

Example :

```
x = 10              # integer
name = "Sagar"      # string
pi = 3.14           # float
is_active = True    # boolean
```

Output (using print) :

```
print(x)
print(name)
print(pi)
print(is_active)
```

Data Types in Python

Data Type	Description	Example
int	Stores integer numbers	10, -5, 100
float	Stores decimal numbers	3.14, 0.5, -2.7
str	Stores text (string)	"Hello", "Python"
bool	Stores True or False	True, False
list	Stores multiple values	[1, 2, 3]
tuple	Immutable collection	(1, 2, 3)
dict	Stores key-value pairs	{ "name": "Sagar" }

Input from User

- input( ) function is used to take input from the user.
- By default, input( ) takes the value as a string.
- We can convert it to other data types if required.

Example :

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Name:", name)
print("Age:", age)
```

Sample Output :

```
Enter your name: Sagar
Enter your age: 19
Name: Sagar
Age: 19
```

@Mastercode.sagar

Introduction to Python Programming

Type Conversion in Python

- Python allows us to convert one data type to another.
- This is called type conversion or type casting.
- We use built-in functions like int(), float(), str(), bool(), etc.
- It is useful when we take input from the user or perform calculations.

Function	Converts To	Example	Result
int()	Integer	int("10")	10
float()	Float	float("3.5")	3.5
str()	String	str(100)	"100"
bool()	Boolean	bool(1)	True

Example (Type Conversion)

```
a = int("10")      # string to integer
b = float("5.6")   # string to float
c = str(25)        # integer to string
d = bool(0)        # integer to boolean
print(a)
print(b)
print(c)
print(d)
```

Output :

```
10
5.6
25
False
```

Taking Multiple Inputs

- We can take multiple inputs from the user in a single line.
- The input() function returns a string, so we may need to convert it to the required data type.
- We can use the split() function to split the input values.

Example (Multiple Inputs)

```
a, b, c = input("Enter three numbers: ").split()
a = int(a)              # converting to integer
b = int(b)
c = int(c)
print("Sum:", a + b + c)
```

Sample Output :

```
Enter three numbers: 5 10 15
Sum: 30
```

@Mastercode.sagar

Introduction to Python Programming

Basic Input and Output in Python

Output in Python

- The print() function is used to display output on the screen.
- We can print text, numbers, or the value of a variable.
- We can also print multiple values separated by commas.

Example

```
print("Hello, World!")
print(10)
print(3.14)
name = "Sagar"
print("My name is", name)
```

Output :

```
Hello, World!
10
3.14
My name is Sagar
```

Input in Python

- The input() function is used to take input from the user.
- By default, input() takes the value as a string.
- We can convert it to other data types like int, float, etc. using type conversion functions.

Example

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Name:", name)
print("Age:", age)
```

Sample Output :

```
Enter your name: Sagar
Enter your age: 19
Name: Sagar
Age: 19
```

Formatted Output

- We can use sep and end parameters in print().

Example

```
print("Python", "is", "easy", sep="-")
print("Hello", end=" ")
print("World!")
```

Output :

```
Python-is-easy
Hello World!
```

@Mastercode.sagar

Introduction to Python Programming

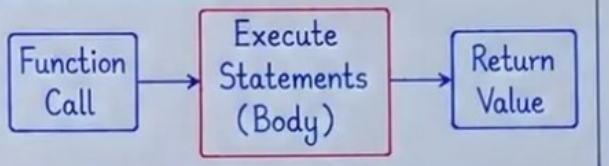
Functions in Python

- Functions are used to group a set of statements and perform a specific task.
- They help in code reusability and make the program modular.
- A function can take input (parameters) and can return a value.
- Python provides built-in functions like print(), len(), type(), etc.
- We can also create our own functions using the def keyword.

Syntax :

```
def function_name(parameters):
    # function body
    return value   # (optional)
```

Function Flow



Example :

```
def add(a, b):
    return a + b

x = add(5, 3)
print("Sum is:", x)
```

Output :

```
Sum is: 8
```

File Handling in Python

- File handling means working with files (read or write data).
- Python provides built-in functions to open, read, write and close files.
- We use the open() function to open a file.
- Modes :

r	- read mode (default)	rb	- read binary
w	- write mode (overwrites)	wb	- write binary
a	- append mode (adds at end)	ab	- append binary

Example :

```
file = open("hello.txt", "w")
file.write("Hello, Python!")
file.close()
```

Result :

A file named hello.txt will be created with content: Hello, Python!

Error Handling in Python

- Errors can occur during program execution (e.g. file not found, divide by zero).
- Python uses try ... except block to handle errors.
- It allows the program to continue running instead of crashing.

Syntax :

```
try:
    # code that may cause error
except Exception as e:
    # handle the error
```

Example :

```
try:
    a = 10 / 0
except Exception as e:
    print("Error:", e)
```

Output :

```
Error: division by zero
```

@Mastercode.sagar

Introduction to Python Programming

Operators in Python

- Operators are used to perform operations on variables and values.
- Python provides different types of operators.
- Operators work with operands (values or variables).

Type	Operators	Description	Example
Arithmetic	+, -, *, /, %, **, //	Used for mathematical operations	5 + 3 = 8
Comparison	==, !=, >, <, >=, <=	Used for comparison	5 > 3 → True
Logical	and, or, not	Used for logical operations	True and False
Assignment	=, +=, -=, *=, /=, %=	Used to assign values	x = 5
Membership	in, not in	Used to check membership	"a" in "name"
Identity	is, is not	Used to check identity	x is y

Arithmetic Operators

- Used to perform basic mathematical operations.
- Operands can be integers, floats or variables.

Example

```
a = 10
b = 3
print(a + b)    # Addition
print(a - b)    # Subtraction
print(a * b)    # Multiplication
print(a / b)    # Division (float)
print(a % b)    # Modulus (remainder)
print(a ** b)   # Exponent (power)
print(a // b)   # Floor division
```

Output

```
13
7
30
3.3333333333333335
1
1000
3
```

Comparison Operators

- Used to compare two values.
- The result is always a boolean value (True or False).

Example

```
x = 5
y = 3
print(x > y)
print(x == y)
print(x != y)
```

Output

```
True
False
True
```

Logical Operators

- Used to combine conditional statements.
- Works with boolean values.

Example

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

Output

```
False
True
False
```

@Mastercode.sagar

Introduction to Python Programming

Loops in Python

- Loops are used to execute a block of code multiple times.
- Python provides two main types of loops: for loop and while loop.
- Loops are useful when we need to repeat a task.

1. for loop

- Used to iterate over a sequence (like list, tuple, string, range, etc.).
- Executes a block of code for each item in the sequence.

Example

```
for i in range(1, 6):
    print(i)
```

Output

```
1
2
3
4
5
```

2. while loop

- Used to repeat a block of code as long as the condition is True.
- The loop continues until the condition becomes False.

Example

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

Output

```
1
2
3
4
5
```

Loop Control Statements

- Python provides special statements to control the execution of loops.

Statement	Description	Example
break	Terminates the loop immediately.	`if i == 3:` `  break`
continue	Skips the current iteration and continues with the next iteration.	`if i == 3:` `  continue`
pass	Does nothing. Used as a placeholder.	`if x > 0:` `  pass`

Nested Loops

- A loop inside another loop is called a nested loop.
- Useful for working with multi-dimensional data (like tables, patterns, etc.).

Example (Print a Pattern)

```
for i in range(1, 4):
    for j in range(1, 4):
        print("*", end=" ")
    print()  # move to next line
```

Output

```
* * *
* * *
* * *
```

@Mastercode.sagar

Introduction to Python Programming

Range Function in Python

- The range() function is used to generate a sequence of numbers.
- It is commonly used in loops.
- It can take 1, 2 or 3 arguments.

Syntax	Description	Example	Output
range(stop)	Generates numbers from 0 to stop-1.	range(5)	0 1 2 3 4
range(start, stop)	Generates numbers from start to stop-1.	range(2, 6)	2 3 4 5
range(start, stop, step)	Generates numbers with a step value.	range(1, 10, 2)	1 3 5 7 9

Example 1

```
for i in range(5):
    print(i)
```

Output

```
0
1
2
3
4
```

Example 2

```
for i in range(2, 6):
    print(i)
```

Output

```
2
3
4
5
```

Example 3

```
for i in range(1, 10, 2):
    print(i)
```

Output

```
1
3
5
7
9
```

Break, Continue and Pass in Loops

- These keywords are used to change the normal flow of a loop.

1. break

- Terminates the loop immediately.
- Control moves to the statement after the loop.

Example

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

Output

```
0
1
2
```

2. continue

- Skips the current iteration.
- Continues with the next iteration.

Example

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

Output

```
0
1
2
4
```

3. pass

- Does nothing.
- Used as a placeholder when a statement is required syntactically.
- Useful when we will add code later.

Example

```
for i in range(5):
    pass
print("Loop finished")
```

Output

```
Loop finished
```

Introduction to Python Programming

Nested Loops in Python

- A loop inside another loop is called a nested loop.
- It is useful for patterns, tables and multi-dimensional data.
- The inner loop runs completely for each iteration of the outer loop.

Example

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j, end=" ")
    print()   # move to next line
```

Output

```
1 1 1 2 1 3
2 1 2 2 2 3
3 1 3 2 3 3
```

Loop with else in Python

- Python allows an else block with loops.
- Else block is executed only when the loop completes normally.
- If the loop is terminated using break, else block is skipped.

Example (Without break)

```
for i in range(3):
    print(i)
else:
    print("Loop completed")
```

Output

```
0
1
2
Loop completed
```

Example (With break)

```
for i in range(3):
    if i == 1:
        break
    print(i)
else:
    print("Loop completed")
```

Output

```
0
```

Infinite Loop

- A loop that never ends is called an infinite loop.
- It keeps running until the program is stopped manually.
- Be careful while using infinite loops.

Example

```
while True:
    print("This will run forever")
```

Output

```
This will run forever
This will run forever
This will run forever
...
```

Introduction to Python Programming

Looping with Different Data Types

- We can use loops with different data types like string, list, tuple, etc.
- Each item is accessed one by one in the loop.
- The loop runs for the length of the data type.

Example 1 : String

```
name = "Python"
for ch in name:
    print(ch)
```

Output

```
P
y
t
h
o
n
```

Example 2 : List

```
fruits = ["apple", "banana",
          "mango"]
for fruit in fruits:
    print (fruit)
```

Output

```
apple
banana
mango
```

Example 3 : Tuple

```
nums = (10, 20, 30)
for n in nums:
    print(n)
```

Output

```
10
20
30
```

Example 4 : Range with Step

```
for i in range(0, 10, 2):
    print(i)
```

Output

```
0
2
4
6
8
```

Using else with Loops

- Python allows an else block with for and while loops.
- The else block runs when the loop completes normally (i.e., not terminated by break).

Example : for loop with else

```
for i in range(3):
    print(i)
else:
    print("Loop executed successfully")
```

Output

```
0
1
2
Loop executed
successfully
```

Example : while loop with else

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
else:
    print("While loop completed")
```

Output

```
1
2
3
While loop
completed
```

Practical Example : Sum of Numbers using for loop

Example

```
sum = 0
for i in range(1, 6):
    sum = sum + i
print("Sum =", sum)
```

Output

```
Sum = 15
```

Explanation

- Loop runs from 1 to 5.
- In each iteration, value of i is added to sum.
- Finally, the total sum is printed.

Introduction to Python Programming

Practical Example : Factorial using for loop

- Factorial of a number is the product of all positive integers up to that number.
- Example : 5! = 5 × 4 × 3 × 2 × 1 = 120

Example

```
n = 5
fact = 1
for i in range(1, n + 1):
    fact = fact * i
print("Factorial =", fact)
```

Output

```
Factorial = 120
```

Explanation

- Loop runs from 1 to n.
- In each iteration, i is multiplied with fact.
- Finally, factorial value is printed.

Practical Example : Check Prime Number using for loop

- A number is prime if it is greater than 1 and divisible only by 1 and itself.
- If it is divisible by any other number, it is not a prime number.

Example

```
n = 7
is_prime = True
if n <= 1:
    is_prime = False
else:
    for i in range(2, n):
        if n % i == 0:
            is_prime = False
            break
if is_prime:
    print("Prime Number")
else:
    print("Not a Prime Number")
```

Output

```
Prime Number
```

Explanation

- If n is less than or equal to 1, it is not prime.
- Loop runs from 2 to n-1.
- If n is divisible by any number in this range, it is not prime.
- Otherwise, it is a prime number.

Practical Example : Sum of Even Numbers using while loop

- This program calculates the sum of even numbers from 1 to 10.
- We use a while loop and keep adding only even numbers.

Example

```
i = 1
sum = 0
while i <= 10:
    if i % 2 == 0:
        sum = sum + i
    i = i + 1
print("Sum of Even Numbers =", sum)
```

Output

```
Sum of Even Numbers = 30
```

Explanation

- Loop runs from 1 to 10.
- In each iteration, we check if the number is even (i % 2 == 0).
- If even, it is added to sum.
- Finally, the total sum is printed.

@Mastercode.sagar

Introduction to Python Programming

Practical Example : Reverse a Number using while loop

- This program reverses a given number.
- We extract the last digit and add it to a new number.
- We remove the last digit in each iteration using floor division (//).

Example

```
n = int(input("Enter a number: "))
rev = 0
while n > 0:
    digit = n % 10
    rev = rev * 10 + digit
    n = n // 10
print("Reversed Number =", rev)
```

Output

```
Enter a number: 1234
Reversed Number = 4321
```

Explanation

- The last digit is obtained using n % 10.
- We build the reversed number using rev = rev * 10 + digit.
- We remove the last digit using n // 10.
- The loop runs until n becomes 0.

Practical Example : Count Digits in a Number using while loop

- This program counts the total number of digits in a given number.
- We repeatedly divide the number by 10 until it becomes 0.

Example

```
n = int(input("Enter a number: "))
count = 0
while n > 0:
    n = n // 10
    count = count + 1
print("Total Digits =", count)
```

Output

```
Enter a number: 5607
Total Digits = 4
```

Explanation

- In each iteration, n is divided by 10 to remove the last digit.
- We increase count by 1 for each digit.
- The loop runs until n becomes 0.
- Finally, total digits are printed.

Practical Example : Find the Sum of Digits using while loop

- This program calculates the sum of all digits in a given number.
- We extract each digit and add it to sum.

Example

```
n = int(input("Enter a number: "))
sum = 0
while n > 0:
    digit = n % 10
    sum = sum + digit
    n = n // 10
print("Sum of Digits =", sum)
```

Output

```
Enter a number: 5678
Sum of Digits = 26
```

Explanation

- The last digit is obtained using n % 10.
- We add each digit to sum.
- We remove the last digit using n // 10.
- The loop runs until n becomes 0.
- Finally, the sum of digits is printed.

Introduction to Python Programming

Using break with Nested Loops

- The break statement terminates only the inner loop.
- The outer loop continues to execute.
- Useful when we want to exit the inner loop based on a condition.

Example

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            break
        print(i, j)
    print("End of inner loop")
```

Output

```
1 1
End of inner loop
2 1
End of inner loop
3 1
End of inner loop
```

Explanation

- When j == 2, the inner loop stops.
- Only j = 1 is printed for each value of i.
- The outer loop continues for i = 1, 2, 3.

Using continue with Nested Loops

- The continue statement skips the current iteration of the inner loop.
- The outer loop continues as normal.
- Useful when we want to skip a particular value.

Example

```
for i in range(1, 4):
    for j in range(1, 4):
        if j == 2:
            continue
        print(i, j)
    print("End of inner loop")
```

Output

```
1 1
1 3
End of inner loop
2 1
2 3
End of inner loop
3 1
3 3
End of inner loop
```

Explanation

- When j == 2, the current iteration is skipped.
- j = 1 and j = 3 are printed for each value of i.
- The outer loop continues normally for i = 1, 2, 3.

Using pass with Nested Loops

- The pass statement does nothing.
- It is used as a placeholder when a statement is required syntactically.
- The loop continues to run as normal.

Example

```
for i in range(1, 3):
    for j in range(1, 4):
        if j == 2:
            pass  # do nothing
        print(i, j)
    print("End of inner loop")
```

Output

```
1 1
1 2
1 3
End of inner loop
2 1
2 2
2 3
End of inner loop
```

Explanation

- When j == 2, pass does nothing.
- The loop continues and all values are printed.
- Useful when we plan to add code later.

Introduction to Python Programming

Practical Example : Print Different Patterns using Nested Loops

- Nested loops are very useful to print patterns.
- We control rows using the outer loop and columns using the inner loop.
- Here are some common patterns.

Example 1 : Right Angle Triangle(Star)

```
for i in range(1, 6):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Example 2 : Inverted Right Angle

```
for i in range(5, 0, -1):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Example 3 : Square Pattern

```
for i in range(1, 5):
    for j in range(1, 5):
        print("*", end=" ")
    print()
```

Output

```
* * * *
* * * *
* * * *
* * * *
```

Example 4 : Rectangle Pattern

```
rows = 3
cols = 6
for i in range(1, rows + 1):
    for j in range(1, cols + 1):
        print("*", end=" ")
    print()
```

Output

```
* * * * * *
* * * * * *
* * * * * *
```

Example 5 : Number Pattern

```
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Example 6 : Inverted Number Pattern

```
for i in range(5, 0, -1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Example 7 : Pyramid Pattern

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n - i + 1):
        print(" ", end=" ")  # spaces
    for k in range(1, 2 * i):
        print("*", end=" ")
    print()
```

Output

```
    *
   * *
  * * *
 * * * * *
* * * * * * *
```

Example 8 : Inverted Pyramid

```
n = 5
for i in range(n, 0, -1):
    for j in range(1, n - i + 1):
        print(" ", end=" ")
    for k in range(1, 2 * i):
        print("*", end=" ")
    print()
```

Output

```
* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *
```

Introduction to Python Programming

More Pattern Examples using Nested Loops

- We can create many creative patterns using nested loops.
- These patterns are useful for practice and to understand loop logic.
- Some more important pattern examples are given below.

Example 1 : Hollow Square Pattern

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == 1 or i == n or
           j == 1 or j == n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
* * * * *
*       *
*       *
*       *
* * * * *
```

Example 2 : Hollow Triangle Pattern

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        if i == n or j == 1 or
           j == i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
*
* *
*   *
*     *
* * * * *
```

Example 3 : Diamond Pattern

```
n = 5
# upper half
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    print("*" * (2 * i - 1))
# lower half
for i in range(n - 1, 0, -1):
    print(" " * (n - i), end="")
    print("*" * (2 * i - 1))
```

Output

```
    *
  * * *
* * * * *
  * * *
    *
```

Example 4 : Hourglass Pattern

```
n = 5
for i in range(n, 0, -1):
    print(" " * (n - i), end="")
    print("*" * (2 * i - 1))
for i in range(2, n + 1):
    print(" " * (n - i), end="")
    print("*" * (2 * i - 1))
```

Output

```
* * * * *
  * * *
    *
  * * *
* * * * *
```

Example 5 : X Pattern

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == j or i + j == n + 1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
*     *
  *  *
    *
  *  *
*     *
```

Example 6 : Plus Pattern

```
n = 5
mid = n // 2 + 1
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == mid or j == mid:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
    *
    *
* * * * *
    *
    *
```

Introduction to Python Programming

More Pattern Examples using Nested Loops (Continued)

- These examples will help us to understand nested loops better.
- We use multiple conditions and variations to print different patterns.
- Such pattern programs are frequently asked in practical exams and interviews.

Example 7 : Triangle Number Pattern

```
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Example 8 : Inverted Number Pattern

```
for i in range(5, 0, -1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Example 9 : Floyd's Triangle

```
num = 1
for i in range(1, 6):
    for j in range(1, i + 1):
        print(num, end=" ")
        num += 1
    print()
```

Output

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Example 10 : Pascal's Triangle

```
n = 5
for i in range(n):
    print(" "*(n-i-1), end="")
    for j in range(i + 1):
        print("*", end=" ")
    print()
```

Output

```
    *
   * *
  * * *
 * * * *
* * * * *
```

Example 11 : Hollow Triangle

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        if i == 1 or i == n or
           j == 1 or j == i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
    *
   * *
  *   *
 *     *
* * * * *
```

Example 12 : Inverted Hollow Triangle

```
n = 5
for i in range(n, 0, -1):
    for j in range(1, i + 1):
        if i == n or i == 1 or
           j == 1 or j == i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
* * * * *
 *     *
  *   *
   * *
    *
```

Example 13 : Diamond Pattern

```
n = 5
for i in range(1, n + 1):
    print(" "*(n-i), end="")
    print("*"*(2*i-1))
for i in range(n-1, 0, -1):
    print(" "*(n-i), end="")
    print("*"*(2*i-1))
```

Output

```
    *
   ***
  *****
 *******
  *****
   ***
    *
```

Example 14 : Butterfly Pattern

```
n = 5
for i in range(1, n + 1):
    print("*"*i, end="")
    print(" "*(2*(n-i)), end="")
    print("*"*i)
for i in range(n, 0, -1):
    print("*"*i, end="")
    print(" "*(2*(n-i)), end="")
    print("*"*i)
```

Output

```
*         *
**       **
***     ***
****   ****
**********
****   ****
***     ***
**       **
*         *
```

Introduction to Python Programming

More Pattern Examples using Nested Loops (Continued)

- These examples cover some special and important patterns.
- You can modify the logic to create many new patterns.
- Practice these programs to improve your problem solving skills.

Example 23 : Plus Pattern

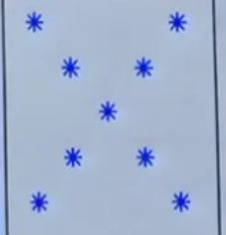
```
n = 5
for i in range(n):
    for j in range(n):
        if i == n//2 or j == n//2:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

Example 24 : Cross Pattern

```
n = 5
for i in range(n):
    for j in range(n):
        if i == j or i + j == n-1:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output



Example 25 : Diamond Outline

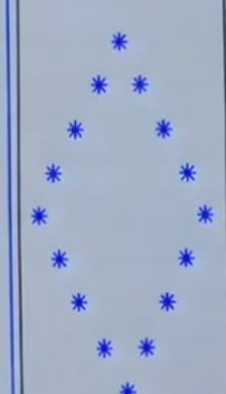
```
n = 5
for i in range(n):
    for j in range(n - i - 1):
        print(" ", end="")
    for j in range(2 * i + 1):
        if j == 0 or j == 2 * i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
for i in range(n - 2, -1, -1):
    for j in range(n - i - 1):
        print(" ", end="")
    for j in range(2 * i + 1):
        if j == 0 or j == 2 * i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

Example 26 : Hollow Diamond

```
n = 5
for i in range(n):
    for j in range(n - i - 1):
        print(" ", end="")
    for j in range(2 * i + 1):
        if j == 0 or j == 2 * i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
for i in range(n - 2, -1, -1):
    for j in range(n - i - 1):
        print(" ", end="")
    for j in range(2 * i + 1):
        if i == 0 or j == 2 * i:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output



Example 27 : Arrow Pattern (Up)

```
n = 5
for i in range(n):
    for j in range(n - i - 1):
        print(" ", end="")
    for j in range(2 * i + 1):
        print("*", end="")
    print()
for i in range(n):
    for j in range(n - 1):
        print(" ", end="")
```

Output

Example 28 : Arrow Pattern (Down)

```
n = 5
for i in range(n, 0, -1):
    for j in range(n - i):
        print(" ", end="")
    for j in range(2 * i - 1):
        print("*", end="")
    print()
for i in range(n):
    for j in range(n - 1):
        print(" ", end="")
```

Output

Introduction to Python Programming

More Pattern Examples using Nested Loops (Continued)

- Here are some more commonly asked pattern programs.
- These patterns improve our understanding of nested loops and space handling.
- Try to write these programs on your own for better practice.

Example 15 : Left Aligned Triangle

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print("*", end="")
    print()
```

Output

```
*
**
***
****
*****
```

Example 16 : Right Aligned Triangle

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n - i + 1):
        print(" ", end="")
    for j in range(1, i + 1):
        print("*", end="")
    print()
```

Output

```
    *
   **
  ***
 ****
*****
```

Example 17 : Inverted Left Triangle

```
n = 5
for i in range(n, 0, -1):
    for j in range(1, i + 1):
        print("*", end="")
    print()
```

Output

```
*****
****
***
**
*
```

Example 18 : Inverted Right Triangle

```
n = 5
for i in range(n, 0, -1):
    for j in range(1, n - i + 1):
        print(" ", end="")
    for j in range(1, i + 1):
        print("*", end="")
    print()
```

Output

```
*****
 ****
  ***
   **
    *
```

Example 19 : Hollow Square

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n + 1):
        if i == 1 or i == n or \
           j == 1 or j == n:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
*****
*   *
*   *
*   *
*****
```

Example 20 : Hollow Rectangle

```
rows = 4
cols = 7
for i in range(1, rows + 1):
    for j in range(1, cols + 1):
        if i == 1 or i == rows or \
           j == 1 or j == cols:
            print("*", end="")
        else:
            print(" ", end="")
    print()
```

Output

```
*******
*     *
*     *
*******
```

Example 21 : Number Pyramid

```
n = 5
for i in range(1, n + 1):
    for j in range(1, n - i + 1):
        print(" ", end="")
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
```

Example 22 : Inverted Number Pyramid

```
n = 5
for i in range(n, 0, -1):
    for j in range(1, n - i + 1):
        print(" ", end="")
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output

```
1 2 3 4 5
 1 2 3 4
  1 2 3
   1 2
    1
```

File Handling in Python

1. Introduction

- File handling allows us to work with files on our computer.
- We can read data from a file, write data to a file, update existing data and also delete files.
- In Python, file handling is very simple using built-in functions.
- Files are used to store data permanently.

2. Types of Files

- Text File (.txt) → stores data in readable text format.
- Binary File (.jpg, .png, .pdf, .exe, etc.) → stores data in binary format.

3. Opening a File

```
f = open("filename", "mode")
```

- filename → name of the file (with path if needed)
- mode → specifies what operation to perform

4. File Modes

Mode	Description
r	Open file for reading (default)
w	Open file for writing (creates new file or overwrites existing file)
a	Open file for appending (adds data at the end)
x	Create a new file (fails if file already exists)
b	Open file in binary mode
t	Open file in text mode (default)

5. Example : Writing to a File

```
f = open("demo.txt", "w")
f.write("Hello, This is Python!")
f.write("\nFile Handling Example.")
f.close()
```

6. Example : Reading from a File

```
f = open("demo.txt", "r")
data = f.read()
print(data)
f.close()
```

7. Example : Appending to a File

```
f = open("demo.txt", "a")
f.write("\nThis line is appended.")
f.close()
```

8. Example : Reading Line by Line

```
f = open("demo.txt", "r")
for line in f:
    print(line, end="")
f.close()
```

9. Example : Using with Statement

```
with open("demo.txt", "r") as f:
    data = f.read()
    print(data)
```

- The with statement automatically closes the file.
- It is a recommended and safer way.

10. Useful File Functions

Function	Description
read()	Reads the entire file
readline()	Reads one line at a time
readlines()	Reads all lines and returns a list
write()	Writes data to a file
writelines()	Writes a list of strings to a file
close()	Closes the file

File Handling in Python (Continued)

11. Example : Writing Multiple Lines

```
f = open("demo.txt", "w")
f.write("Python\n")
f.write("File Handling\n")
f.write("Example\n")
f.close()
```

Output (demo.txt)

```
Python
File Handling
Example
```

12. Example : Reading All Lines

```
f = open("demo.txt", "r")
lines = f.readlines()
for line in lines:
    print(line, end="")
f.close()
```

Output

```
Python
File Handling
Example
```

13. Example : Counting Lines

```
f = open("demo.txt", "r")
count = 0
for line in f:
    count += 1
print("Total lines:", count)
f.close()
```

Output

```
Total lines: 3
```

14. Example : Appending Multiple Lines

```
f = open("demo.txt", "a")
f.write("New Line 1\n")
f.write("New Line 2\n")
f.close()
```

Output (demo.txt)

```
Python
File Handling
Example
New Line 1
New Line 2
```

15. Example : Reading Specific Number of Characters

```
f = open("demo.txt", "r")
data = f.read(7)
print(data)
f.close()
```

Output

```
Python
```

16. Example : Using readlines() with Loop

```
f = open("demo.txt", "r")
for line in f.readlines():
    print(line.strip())
f.close()
```

Output

```
Python
File Handling
Example
New Line 1
New Line 2
```

17. Example : Checking if File Exists

```
import os
if os.path.exists("demo.txt"):
    print("File exists")
else:
    print("File not found")
```

Output

```
File exists
```

18. Example : Deleting a File

```
import os
if os.path.exists("sample.txt"):
    os.remove("sample.txt")
    print("File deleted")
else:
    print("File not found")
```

Output

```
File deleted
```

19. Example : Renaming a File

```
import os
if os.path.exists("old.txt"):
    os.rename("old.txt", "new.txt")
    print("File renamed")
else:
    print("File not found")
```

Output

```
File renamed
```

20. Important Points

1. Always close the file after use (or use with statement).
2. Use appropriate file mode (r, w, a, etc.).
3. Use exception handling to avoid errors.
4. Do not open large files unnecessarily in read()
5. Use with open(...) as f: for better practice.
6. Always check if file exists before reading or deleting.

Exception Handling in Python

1. Introduction

- An exception is an error that occurs during the execution of a program.
- Exception handling allows us to handle errors gracefully without stopping the program.
- It helps to maintain the normal flow of the program.

2. Syntax of Exception Handling

```
try:
    # code that may raise an exception
except ExceptionType:
    # code to handle the exception
else:
    # code if no exception occurs
finally:
    # code that always executes
```

3. Example : Division by Zero

```
try:
    a = 10
    b = 0
    c = a / b
    print("Result:", c)
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
```

Output

```
Error: Cannot divide by zero
```

4. Example : Invalid Input

```
try:
    n = int(input("Enter a number: "))
    print("You entered:", n)
except ValueError:
    print("Error: Please enter a valid number")
```

Output

```
Enter a number: a
Error: Please enter a valid number
```

5. Example : Multiple Except Blocks

```
try:
    x = int(input("Enter a number: "))
    result = 10 / x
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero")
except ValueError:
    print("Error: Invalid input")
```

Output

```
Enter a number: 0
Error: Cannot divide by zero
```

6. Example : Using else Block

```
try:
    n = int(input("Enter a number: "))
    print("Number:", n)
except ValueError:
    print("Invalid input")
else:
    print("No error occurred")
```

Output

```
Enter a number: 5
Number: 5
No error occurred
```

7. Example : Using finally Block

```
try:
    f = open("demo.txt", "r")
    data = f.read()
    print(data)
except FileNotFoundError:
    print("File not found")
finally:
    print("This block always executes")
```

Output

```
File not found
This block always executes
```

8. Example : Raising an Exception

```
age = int(input("Enter your age: "))
if age < 0:
    raise ValueError("Age cannot be negative")
else:
    print("Your age is:", age)
```

Output

```
Enter your age: -5
Traceback (most recent call last):
ValueError: Age cannot be negative
```

9. Example : Custom Exception

```
class NegativeNumberError(Exception):
    pass

n = int(input("Enter a number: "))
if n < 0:
    raise NegativeNumberError("Negative numbers are not allowed")
else:
    print("You entered:", n)
```

Output

```
Enter a number: -3
Traceback (most recent call last):
NegativeNumberError:
Negative numbers are not allowed
```

10. Important Points

1. Use try-except to handle errors.
2. Use specific exception types instead of generic except.
3. Use else block for code that runs when no exception occurs.
4. Use finally block for cleanup operations (e.g., closing files).
5. You can also create custom exceptions.